TECHNICAL DOCUMENTATION

DAM Intermodular Project

Cycle: Multiplatform Application Development (DAM)

Course: 2025/2026

Application: **BUBBLE** - Social network based on Geo-location

Author: [Iker Martinez Lago]

Date: March 2026

📑 Table of Contents

1. System Architecture
2. API
3. Database
4. Web Application
5. Mobile App
6. Desktop Application
7. Security
8. Deployment
9. Tests
10. Use and Maintenance Guide
11. Future Improvements

---

1. System Architecture

1.1 General Scheme

The **BUBBLE** application is a three-layer distributed system:

```
┌──────────────────────────────────────────────────────────┐
│                    CAPA DE PRESENTACIÓN                  │
├──────────────────────┬──────────────────┬────────────────┤
│   Android (Móvil)    │  Desktop (.NET)  │   Web (Laravel)│
│                      │                  │                │
│ • Google Maps API    │ • Windows Forms  │ • HTML/CSS     │
│ • Location Services  │ • Custom Controls│ • Api Laravel  │
│ • Kotlin/Java        │ • C# / .NET 4.7  │                │
└──────────┬───────────┴────────┬─────────┴────────┬───────┘
           │                    │                  │
           └────────┬───────────┴────────┬─────────┘
                    │                    │
                    └────────┬───────────┘
                             │
                             │ REST API (HTTP)
                             ▼
        ┌──────────────────────────────────────────┐
        │      CAPA DE LÓGICA DE NEGOCIO            │
```

```
        │         Laravel 12 Backend API             │
        │                                            │
        │ • Controllers (API + Web)                  │
        │ • Models (Eloquent ORM)                    │
        │ • Middleware (Autenticación Sanctum/Web)   │
        │ • Validaciones                             │
        └────────────────────┬────────────────────────┘
                             │
                             ▼
        ┌──────────────────────────────────────────┐
        │        CAPA DE PERSISTENCIA               │
        │      SQLite Database (database.sql)       │
        │                                            │
        │ • Tabla users                              │
        │ • Tabla bubbles                            │
        │ • Tabla personal_access_tokens             │
        │ • Tabla sessions                           │
        └──────────────────────────────────────────┘
```

1.2 Main Technologies

Backend (API + Web Integrated in Laravel)

- **Framework**: Laravel 12
- **Language**: PHP 8.2+
- **Database**: SQLite (development) / MySQL/PostgreSQL (production)
- **ORM**: Eloquent
- **Authentication**:
  - **Web**: Session-based (guard web)
  - **API**: Laravel Sanctum (tokens)

Frontend Web (Integrated in Laravel)

- **Templating**: Blade (SSR)
- **CSS**: Tailwind CSS v4.0.0
- **JavaScript**: Vanilla ES6+ + Axios 1.11.0
- **Maps**: Google Maps API v3
- **Build Tool**: Vite 7.0.7
- **Package Manager**: npm
- **Server**: Apache/Nginx (Vite for development)

Mobile Application

- **Platform**: Android
- **Language**: Kotlin/Java
- **Minimum SDK**: Android 7.0 (API 24)
- **Target SDK**: Android 15 (API 36)
- **Build System**: Gradle
- **Key Libraries**:
  - Google Maps API (v18.2.0)
  - Google Play Services Location (v21.0.1)
  - Glide (v4.16.0) for image upload
  - BCrypt (v0.4) for password hashing

Desktop Application

- **Platform**: Windows
- **Language**: C#
- **Framework**: .NET Framework 4.7.2
- **UI**: Windows Forms (WinForms)
- **Custom Components**: Radar Map Control
- **WebView**: Microsoft.Web.WebView2 (v1.0.2903.40)

1.3 Communication Flow

```
┌──────────────────┐
│  Cliente Móvil   │
│    (Android)     │
└──────────────────┘
         │
         │
         │ HTTP/REST
         │
         ▼
    ┌──────────────────────────┐
    │  API Laravel Backend     │
    │ (bubble-api)             │
    └──────────────────────────┘
         ▲                 ▲
         │                 │
         │ HTTP/REST       │ SQL
         │                 │
         │                 ▼
         │           ┌──────────────┐
         │           │ SQLite DB    │
         │           │ (database.   │
         │           │  sqlite)     │
         │           └──────────────┘
         │
         │
    ┌──────────────────────┐
    │  Cliente Escritorio  │
    │  (Windows .NET)      │
    └──────────────────────┘
```

---

2. API

2.1 General Description

The RESTful API is built with Laravel 12 and provides all the services needed to:

- Authentication and user management
- Create, list, update and delete "bubbles" (geo-located messages)
- User profile management

Base URL: `http://localhost:8000/api` (local development)

2.2 Authentication

The API uses **Laravel Sanctum** for token-based authentication.

Types of Routes

- **Public**: Without authentication (login, registration, list bubbles)
- **Protected**: Requires Sanctum token in header `Authorization: Bearer {TOKEN}`

Authentication Flow

```
1. Cliente POST a /desktop/login
   ├─ Email + Password
   └─> Respuesta: {token, user}

2. Cliente guarda token

3. Cliente incluye en próximas requests:
   ├─ Header: Authorization: Bearer {token}
   └─> API valida y permite acceso

4. Logout: POST a /desktop/logout
   └─> Token se invalida
```

2.3 Endpoint List

Authentication (Desktop)

Endpoint	Method	Auth	Description
`/desktop/login`	POST	No	User login
`/desktop/register`	POST	No	User registration
`/desktop/logout`	POST	Yeah	Sign out
`/desktop/profile`	GET	Yeah	Get user profile
`/desktop/profile`	POST	Yeah	Update profile

Bubbles

Endpoint	Method	Auth	Description
`/burbujas`	GET	No	List all bubbles
`/burbujas`	POST	No	Create new bubble
`/desktop/bubble`	POST	Yeah	Create bubble (authenticated user)
`/desktop/bubble`	DELETE	Yeah	Delete own bubble

Dashboard

Endpoint	Method	Auth	Description

`/desktop/dashboard`	GET	Yeah	Get user dashboard

2.4 Examples of Requests and Responses

2.4.1 User Registration

Request:

```
POST /api/desktop/register HTTP/1.1
Host: localhost:8000
Content-Type: application/json

{
  "name": "Iker Martinez",
  "email": "iker@example.com",
  "password": "SecurePass123!",
  "password_confirmation": "SecurePass123!"
}
```

Successful Response (201):

```
{
  "message": "User registered successfully",
  "user": {
    "id": 1,
    "name": "Iker Martinez Lago",
    "email": "iker@example.com",
    "avatar": null
  },
  "token": "1|abc123def456..."
}
```

Error Response (422):

```
{
  "message": "The given data was invalid.",
  "errors": {
    "email": ["The email has already been taken."],
    "password": ["The password must be at least 8 characters."]
  }
}
```

2.4.2 Login

Request:

```
POST /api/desktop/login HTTP/1.1
Host: localhost:8000
Content-Type: application/json
```

```
{
  "email": "iker@example.com",
  "password": "SecurePass123!"
}
```

Successful Response (200):

```
{
  "message": "Login successful",
  "user": {
    "id": 1,
    "name": "Iker Martinez Lago",
    "email": "iker@example.com",
    "avatar": null
  },
  "token": "1|abc123def456..."
}
```

Error Response (401):

```
{
  "message": "Invalid credentials"
}
```

2.4.3 Create Bubble

Request:

```
POST /api/desktop/bubble HTTP/1.1
Host: localhost:8000
Authorization: Bearer 1|abc123def456...
Content-Type: application/json

{
  "latitude": 40.4168,
  "longitude": -3.7038,
  "mensaje": "¡Qué día tan bonito en Madrid!"
}
```

Successful Response (201):

```
{
  "message": "Bubble created successfully",
  "bubble": {
    "id": 42,
    "user_id": 1,
    "latitude": 40.4168,
    "longitude": -3.7038,
    "mensaje": "¡Qué día tan bonito en Madrid!"
```

```
  }
}
```

2.4.4 List Bubbles

Request:

```
GET /api/burbujas HTTP/1.1
Host: localhost:8000
```

Answer (200):

```
{
  "message": "Bubbles retrieved successfully",
  "bubbles": [
    {
      "id": 1,
      "user_id": 1,
      "latitude": 40.4168,
      "longitude": -3.7038,
      "mensaje": "¡Qué día tan bonito en Madrid!",
      "user": {
        "id": 1,
        "name": "Iker Martinez Lago",
        "email": "iker@example.com"
      }
    }
  ]
}
```

2.4.5 Get Profile

Request:

```
GET /api/desktop/profile HTTP/1.1
Host: localhost:8000
Authorization: Bearer 1|abc123def456...
```

Answer (200):

```
{
  "user": {
    "id": 1,
    "name": "Iker Martinez Lago",
    "email": "iker@example.com"
  }
}
```

2.4.6 Update Profile

Request:

```
POST /api/desktop/profile HTTP/1.1
Host: localhost:8000
Authorization: Bearer 1|abc123def456...
Content-Type: application/json

{
  "name": "Iker Martinez Lago",
  "avatar": "https://example.com/avatar.jpg"
}
```

Answer (200):

```
{
  "message": "Profile updated successfully",
  "user": {
    "id": 1,
    "name": "Iker Martinez Lago",
    "email": "iker@example.com",
    "avatar": "https://example.com/avatar.jpg"
  }
}
```

2.4.7 Logout

Request:

```
POST /api/desktop/logout HTTP/1.1
Host: localhost:8000
Authorization: Bearer 1|abc123def456...
```

Answer (200):

```
{
  "message": "Logged out successfully"
}
```

2.5 Error Codes

Code	Meaning	Cause
400	Bad Request	Invalid data in the request
401	Unauthorized	Invalid or expired token
403	Forbidden	User does not have permission
404	Not Found	Resource not found
422	Unprocessable Entity	Data validation failed

500	Internal Server Error	Server error

---

3. Database

3.1 Description

The database uses **SQLite** with Laravel **Eloquent ORM** for model and relationship management.

Location: `bd/bubble-api/database/database.sqlite`

3.2 Entity-Relationship Diagram

```
┌──────────────────────────┐
│       USERS              │
├──────────────────────────┤
│ id (PK)                  │
│ name                     │
│ email (UNIQUE)           │
│ email_verified_at        │
│ password (hashed)        │
│ avatar                   │
│ remember_token           │
│ created_at               │
│ updated_at               │
└──────────────┬───────────┘
               │
               │ 1:N
               │
               ▼
┌──────────────────────────┐
│      BUBBLES             │
├──────────────────────────┤
│ id (PK)                  │
│ user_id (FK) ───┐        │
│ latitude        │        │
│ longitude       │        │
│ mensaje         │        │
│ created_at      │        │
│ updated_at      │        │
└─────────────────┴────────┘
                  │
                  └─ Relación: User hasMany Bubble
                     Bubble belongsTo User

┌──────────────────────────────────────┐
│ PERSONAL_ACCESS_TOKENS (Sanctum)     │
├──────────────────────────────────────┤
│ id (PK)                              │
│ tokenable_type                       │
│ tokenable_id                         │
│ name                                 │
```

```
│ token (hashed)                        │
│ abilities                             │
│ last_used_at                          │
│ created_at                            │
│ updated_at                            │
└───────────────────────────────────────┘

┌──────────────────────────┐
│      SESSIONS            │
├──────────────────────────┤
│ id (PK)                  │
│ user_id (nullable)       │
│ ip_address               │
│ user_agent               │
│ payload                  │
│ last_activity            │
│ created_at               │
│ updated_at               │
└──────────────────────────┘

┌──────────────────────────────┐
│  PASSWORD_RESET_TOKENS       │
├──────────────────────────────┤
│ email (PK)                   │
│ token (hashed)               │
│ created_at                   │
└──────────────────────────────┘

┌──────────────────────────┐
│        CACHE             │
├──────────────────────────┤
│ key (PK)                 │
│ value                    │
│ expiration               │
└──────────────────────────┘

┌──────────────────────────┐
│        JOBS              │
├──────────────────────────┤
│ id (PK)                  │
│ queue                    │
│ payload                  │
│ attempts                 │
│ reserved_at              │
│ available_at             │
│ created_at               │
└──────────────────────────┘
```

3.3 Description of Tables

3.3.1 Table `users`

Field	Guy	Restrictions	Description
`id`	INT	PK, AUTO_INCREMENT	Unique identifier
`name`	VARCHAR(255)	NOT NULL	User name
`email`	VARCHAR(255)	NOT NULL, UNIQUE	Unique email
`email_verified_at`	TIMESTAMP	NULLABLE	Email verification
`password`	VARCHAR(255)	NOT NULL	Hashed Password (BCrypt)
`avatar`	VARCHAR(255)	NULLABLE	Avatar URL
`remember_token`	VARCHAR(100)	NULLABLE	"remember me" token
`created_at`	TIMESTAMP	NOT NULL	Creation date
`updated_at`	TIMESTAMP	NOT NULL	Last update date

Indices:

- PRIMARY KEY: `id`
- UNIQUE: `email`

Validations:

- Unique and valid email
- Password minimum 8 characters
- Non-empty name

3.3.2 Table `bubbles`

Field	Guy	Restrictions	Description
`id`	INT	PK, AUTO_INCREMENT	Unique identifier
`user_id`	INT	FK → users(id), NOT NULL	Owner user
`latitude`	DECIMAL(10.8)	NOT NULL	Geographic latitude
`longitude`	DECIMAL(11.8)	NOT NULL	geographic longitude
`mensaje`	TEXT	NOT NULL	Message content
`created_at`	TIMESTAMP	NOT NULL	Creation date
`updated_at`	TIMESTAMP	NOT NULL	Last update date

Indices:

- PRIMARY KEY: `id`
- FOREIGN KEY: `user_id` → `users(id)`

Validations:

- Latitude between -90 and 90
- Length between -180 and 180

- Non-empty message (max 1000 characters)
- User must exist

3.3.3 Table `personal_access_tokens` (Sanctum)

Field	Guy	Restrictions	Description
`id`	INT	PK, AUTO_INCREMENT	Unique identifier
`tokenable_type`	VARCHAR(255)	NOT NULL	Model type ("App\Models\User")
`tokenable_id`	INT	NOT NULL	User ID
`name`	VARCHAR(255)	NOT NULL	Token name
`token`	VARCHAR(80)	NOT NULL, UNIQUE	Hashed token
`abilities`	JSON	NULLABLE	Permissions/abilities
`last_used_at`	TIMESTAMP	NULLABLE	Last use
`created_at`	TIMESTAMP	NOT NULL	Creation date
`updated_at`	TIMESTAMP	NOT NULL	Update date

Use: Sanctum API Authentication

3.3.4 Table `sessions`

Field	Guy	Description
`id`	VARCHAR(255)	session ID
`user_id`	INT	User (nullable)
`ip_address`	VARCHAR(45)	IP address
`user_agent`	TEXT	Browser/Client info
`payload`	LONGTEXT	Session data
`last_activity`	INT	Timestamp last access

Use: Traditional web session management

3.4 Eloquent Relationships

```php
// User.php
class User extends Model {
    public function bubbles() {
        return $this->hasMany(Bubble::class);
    }
}

// Bubble.php
class Bubble extends Model {
```

```
    public function user() {
        return $this->belongsTo(User::class);
    }
}
```

3.5 Migrations

Laravel migrations are in `database/migrations/` :

1. **`0001_01_01_000000_create_users_table.php`**
    - Create table `users` , `password_reset_tokens` , `sessions`
2. **`0001_01_01_000001_create_cache_table.php`**
    - Create table `cache` for caching
3. **`0001_01_01_000002_create_jobs_table.php`**
    - Create table `jobs` for job queue
4. **`2026_01_28_114031_create_personal_access_tokens_table.php`**
    - Create table `personal_access_tokens` (Sanctum)
5. **`2026_01_28_120139_create_bubbles_table.php`**
    - Create table `bubbles` in relation to `users`

Execution of migrations:

```
php artisan migrate
```

Revert last migration:

```
php artisan migrate:rollback
```

3.6 Seeders

Seeders available at `database/seeders/` :

- `DatabaseSeeder.php` - Run main seeders

Run seeders:

```
php artisan db:seed
```

---

4. Web Application

4.1 Description

Full-Stack Web Application integrated directly into Laravel as **Single-Page Application (SPA)** with Server Side Rendering (SSR). Implements a geo-referenced location system with an interactive map.

Location: `bd/bubble-api/` (integrated in Laravel)

4.2 Technology

Component	Technology	Version
Backend	Laravel	11+
Backend Language	PHP	8.2+
Frontend Rendering	Blade (SSR)	Native Laravel
CSS	Tailwind CSS	v4.0.0
JavaScript	Vanilla JS + Axios	ES6+
Maps	Google Maps API	v3
Build Tool	Vite	v7.0.7
Package Manager	npm	-

4.3 Project Structure

```
bubble-api/
├── resources/
│   ├── views/
│   │   ├── welcome.blade.php          (Home page)
│   │   ├── login.blade.php            (Login form)
│   │   ├── registro.blade.php         (Registration form)
│   │   ├── muro.blade.php             (Main map interface)
│   │   └── perfil.blade.php           (User profile editor)
│   ├── js/
│   │   ├── app.js                     (Entry point)
│   │   └── bootstrap.js               (Axios configuration)
│   └── css/
│       └── app.css                    (CSS + custom styles)
├── app/Http/Controllers/
│   ├── ControladorRegistro.php        (Auth: login/register/logout)
│   ├── BubbleController.php           (Bubble CRUD + muro view)
│   └── ProfileController.php          (Profile management)
├── routes/
│   ├── web.php                        (Web routes - 17 rutas)
│   └── api.php                        (API routes - desktop/mobile)
├── config/
│   ├── auth.php                       (Session-based auth)
│   └── session.php
├── public/
│   ├── index.php
│   ├── img/
│   │   ├── logo.gif
│   │   └── fondo.gif
│   └── storage/                       (User avatars & uploads)
```

```
├── vite.config.js
├── package.json
└── composer.json
```

4.4 Web Routes

Route	Method	Auth	Controller	Purpose
`/`	GET	No	BubbleController	Home (redirects to login)
`/login`	GET	No	ControllerRegister	Login form
`/login`	POST	No	ControllerRegister	Process login
`/registro`	GET	No	ControllerRegister	Registration form
`/registro`	POST	No	ControllerRegister	Process registration
`/muro`	GET	**Yeah**	BubbleController	Main map (protected)
`/muro`	POST	**Yeah**	BubbleController	Create/update bubble
`/logout`	POST	**Yeah**	ControllerRegister	Sign out
`/perfil`	GET	**Yeah**	ProfileController	Edit profile
`/perfil`	PATCH	**Yeah**	ProfileController	Save profile
`/api/notificaciones`	GET	No	BubbleController	AJAX radar (updates every 2s)

4.5 Main Views

4.5.1 `login.blade.php`

Features:

- Authentication form
- Dark theme design (#131313)
- GIF animated background
- Responsive layout (2 columns on desktop, 1 on mobile)
- Links to recover password and registration

```
<!-- Form structure -->
<form method="POST" action="/login">
  <input type="email" name="email" required>
  <input type="password" name="password" required>
  <button type="submit">Iniciar Sesión</button>
  <label><input type="checkbox" name="remember"> Recuérdame</label>
</form>
```

4.5.2 `registro.blade.php`

Features:

- Registration form

- Blue/purple theme design (#3b4cca)
- Field validation
- Fields: name, email, password

```
<form method="POST" action="/registro">
  <input type="text" name="name" placeholder="Nombre de usuario" required>
  <input type="email" name="email" placeholder="Correo electrónico" required>
  <input type="password" name="password" placeholder="Contraseña" required>
  <input type="password" name="password_confirmation" placeholder="Confirmar contraseña"
required>
  <button type="submit">Registrarse</button>
</form>
```

4.5.3 `muro.blade.php` - Main Screen

Layout Architecture:

```
┌──────────────────────────────────────┐
│           Google Maps                │  3 columnas: sidebar (300px) | mapa | panel perfil
(250px)
│ ● User location (with arrow)         │
│ ◯ Nearby bubbles (distance)          │
│                                      │
│                                      │
└──────────────────────────────────────┘
```

Features:

1. **Left Column (Radar)**
   - List of nearby bubbles
   - Shows: Avatar, name, distance
   - Updates every 2 seconds via AJAX
   - Colors: Green (#2ecc71) for active bubbles
2. **Center (Google Maps)**
   - Interactive map with 17x zoom and 45° tilt
   - User bookmark (current position)
   - Bubble circles/markers
   - Automatic geolocation (`navigator.geolocation.watchPosition()`)
   - Optional traffic layer
3. **Right Column (User Profile)**
   - Authenticated user avatar
   - Name
   - Toggle button to create/delete current bubble
   - Logout button
   - Link to full profile

Main JavaScript:

```
// Geolocalización continua y actualización de mapa
navigator.geolocation.watchPosition((position) => {
  const lat = position.coords.latitude;
  const lng = position.coords.longitude;
  updateMap(lat, lng);
  updateRadar(); // Actualiza lista cada 2s
});

// Toggle crear/eliminar burbuja
document.getElementById('toggle-burbuja').addEventListener('change', (e) => {
  if (e.target.checked) {
    createBubble(currentLat, currentLng);
  } else {
    deleteBubble();
  }
});
```

4.5.4 `perfil.blade.php` - Profile Editor

Features:

- Editing username
- Avatar upload (image)
- Avatar Preview
- Button back to `/muro`
- Server validation

```
<form method="PATCH" action="/perfil" enctype="multipart/form-data">
  <input type="text" name="name" value="{{ auth()->user()->name }}">
  <input type="file" name="avatar" accept="image/*">
  <img id="preview" src="{{ asset('storage/avatars/' . auth()->user()->avatar) }}">
  <button type="submit">Guardar Perfil</button>
</form>
```

4.6 Authentication (Session-Based)

Web Authentication Flow:

```
1. Usuario accede a /
   └> Redirige a /login (middleware web)

2. Usuario ingresa email + password
   └> POST /login
   └> ControladorRegistro::login()
   └> Auth::attempt($credentials)
   └> Session creada (config/session.php)
   └> Redirige a /muro

3. En /muro protegida por middleware('auth')
   └> Auth::user() disponible en Blade
   └> session('user_id') accesible
```

```
4. Logout
   └─> POST /logout
   └─> Auth::logout()
   └─> Session destruida
   └─> Redirige a /login
```

Authentication Guard:

```php
// config/auth.php
'guards' => [
    'web' => [
        'driver' => 'session',
        'provider' => 'users',
    ],
],

'providers' => [
    'users' => [
        'driver' => 'eloquent',
        'model' => App\Models\User::class,
    ],
],
```

4.7 Connection with API

The website uses **server-side sessions**, NOT Sanctum API tokens.

To create/update data, use web routes that process the form:

```php
// POST /muro - Crear burbuja
Route::post('/muro', [BubbleController::class, 'guardar'])->middleware('auth');

// PATCH /perfil - Actualizar perfil
Route::patch('/perfil', [ProfileController::class, 'update'])->middleware('auth');

// POST /logout - Logout
Route::post('/logout', [ControladorRegistro::class, 'logout'])->middleware('auth');
```

JavaScript Client (Axios):

```javascript
// resources/js/bootstrap.js
window.axios = axios.create({
  baseURL: '/',
  headers: {
    'X-Requested-With': 'XMLHttpRequest',
    'X-CSRF-TOKEN': document.querySelector('meta[name="csrf-token"]').content,
  },
});

// Uso en vistas
```

```
fetch('/api/notificaciones') // Retorna HTML para radar
  .then(r => r.text())
  .then(html => document.getElementById('radar').innerHTML = html);
```

4.8 Styles and Design

CSS Framework: Tailwind CSS v4

Color Theme:

- **Main**: #3b4cca (Blue)
- **Background**: #131313 (Dark)
- **Accent**: #2ecc71 (Green)
- **Panels**: rgba(255,255,255,0.9) (Translucent white)
- **Error**: #ff4d4d (Red)

Tailwind configuration (`resources/css/app.css`):

```
@import "tailwindcss";

@theme {
  --color-primary: #3b4cca;
  --color-dark: #131313;
  --color-accent: #2ecc71;
}

.btn-primary {
  @apply px-4 py-2 bg-primary text-white rounded-lg hover:opacity-90;
}

.panel {
  @apply bg-white bg-opacity-90 backdrop-blur rounded-3xl shadow-lg p-6;
}
```

Sources:

- Body: 'Instrument Sans' (Google Fonts)
- Monospace (Profile, Wall): 'Roboto Mono'
- Icons: Font Awesome 6.0 (CDN)

4.9 Build Pipeline (Vite)

Settings (`vite.config.js`):

```
import { defineConfig } from 'vite';
import laravel from 'laravel-vite-plugin';
import tailwindcss from '@tailwindcss/vite';

export default defineConfig({
  plugins: [
    laravel(['resources/css/app.css', 'resources/js/app.js']),
    tailwindcss(),
```

```
  ],
});
```

npm scripts:

```
npm run dev     # Inicia servidor Vite (http://localhost:5173)
npm run build   # Construye para producción (public/build/)
npm run preview # Preview del build
```

Development:

```
# Terminal 1: Laravel
php artisan serve

# Terminal 2: Vite
npm run dev

# Acceder a: http://localhost:8000
```

4.10 Main Functions of Controllers

RegistrationController.php:

- `showLogin()` - Sample login form
- `login()` - Process login (validate email/password)
- `showRegistro()` - Sample registration form
- `registro()` - Creates user and authenticates it
- `logout()` - Sign out

BubbleController.php:

- `mostrarMuro()` - Load main page with map
- `crearBurbuja()` - Create bubble in GPS coordinates
- `eliminarBurbuja()` - Remove user bubble
- `getNotificacionesAjax()` - Return nearby bubbles (AJAX)

4.11 Main Features

1. **Real Time Geolocation**
   - `navigator.geolocation.watchPosition()` continuous
   - High precision enabled
   - Automatic map center
2. **Bubble Radar**
   - AJAX update every 2 seconds
   - Distance calculated with Haversine formula
   - Search radius: 5 km
3. **Bubble Management**
   - Toggle checkbox to activate/deactivate
   - One bubble per user (the previous one is deleted)

- Automatic GPS coordinates

4. **User Profile**
   - Uploadable avatar (stored in `storage/avatars/`)
   - Editable name
   - Fallback to UI-avatars API if no avatar
5. **Interactive Map**
   - Google Maps v3
   - 17x zoom, 45° tilt
   - User marker with direction arrow
   - Circles for nearby bubbles

4.12 Google Maps Settings

Get API Key in Google Cloud Console and include in `muro.blade.php`:

```
<script src="https://maps.googleapis.com/maps/api/js?key=YOUR_API_KEY"></script>

<script>
  var map = new google.maps.Map(document.getElementById('map'), {
    zoom: 17,
    tilt: 45,
    center: {lat: 40.4168, lng: -3.7038}, // Madrid por defecto
  });
</script>
```

---

5. Mobile Application

5.1 Description

Native Android Application to view and create "bubbles" (geo-located messages) in real time.

Path: `Movil/Bubble/`

5.2 Technology

- **Language**: Kotlin/Java
- **Minimum SDK**: Android 7.0 (API 24)
- **Target SDK**: Android 15 (API 36)
- **Build System**: Gradle Kotlin DSL
- **Namespace**: `com.example.bubble`

5.3 Main Dependencies

```
// Google Play Services
implementation("com.google.android.gms:play-services-maps:18.2.0")
implementation("com.google.android.gms:play-services-location:21.0.1")

// Utilities
implementation("org.mindrot:jbcrypt:0.4")           // Password hashing
```

```
implementation("com.github.bumptech.glide:glide:4.16.0") // Image loading
implementation("androidx.coordinatorlayout:coordinatorlayout:1.2.0")

// Material Design
implementation("com.google.android.material:material:1.9.0")
```

5.4 Required Permissions (AndroidManifest.xml)

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
<uses-permission android:name="android.permission.VIBRATE" />
```

5.5 Main Activities

Activity	Purpose	Flow
`Welcome`	Welcome screen	Launcher → Login/Registration
`LogIn`	User login	Credential entry
`Registro`	New user registration	Account creation
`Principal`	Main screen	Map + Bubble List

5.6 Application Flow

```
┌──────────────┐
│   Welcome    │  (Launcher)
└──────┬───────┘
       │
       ├──────────────┬──────────────┐
       │              │              │
       ▼              ▼              ▼
   ┌───────┐      ┌───────┐      ┌───────┐
   │ LogIn │      │Registro│ (Alt)  │Principal│
   └───┬───┘      └───┬───┘      └───▲───┘
       │              │              │
       │              │              │
       └──────┬───────┘              │
              │                      │
              ▼                      │
          ┌──────────────────────┐   │
          │ API Authentication   │   │
          │ (POST /desktop/login)│──┘
          │ (POST /desktop/register)│
          └──────────────────────┘
```

5.7 Main Features

1. **Geolocation**

    - Get current position of the device
    - Google Maps API for visualization
    - Configurable search radius

2. **Authentication**
    - Login/Registration against API
    - Locally stored token (SharedPreferences)
    - Passwords hashed with BCrypt

3. **Bubble Display**
    - Interactive map with markers
    - List of nearby bubbles
    - User information (avatar, name)

4. **Create Bubbles**
    - Form with message and location
    - POST to `/api/desktop/bubble` or `/api/burbujas`

5.8 Project Structure

```
Movil/Bubble/
├── app/
│   └── src/
│       ├── main/
│       │   ├── java/com/example/bubble/
│       │   │   ├── activities/
│       │   │   │   ├── Welcome.kt
│       │   │   │   ├── LogIn.kt
│       │   │   │   ├── Registro.kt
│       │   │   │   └── Principal.kt
│       │   │   ├── models/
│       │   │   │   ├── User.kt
│       │   │   │   └── Bubble.kt
│       │   │   └── services/
│       │   │       └── ApiClient.kt
│       │   ├── res/
│       │   │   ├── layout/
│       │   │   ├── values/
│       │   │   ├── drawable/
│       │   │   └── menu/
│       │   └── AndroidManifest.xml
│       └── test/
├── build.gradle.kts
└── gradle.properties
```

5.9 Google Maps Settings

1. Generate API Key in Google Cloud Console
2. Add to `AndroidManifest.xml`:

```
<meta-data
    android:name="com.google.android.geo.API_KEY"
    android:value="YOUR_API_KEY_HERE" />
```

---

6. Desktop Application

6.1 Description

Native Windows Application in .NET Framework for bubble management with complete graphical interface.

Path: `Escritorio/BubbleApp/`

6.2 Technology

- **Language**: C#
- **Framework**: .NET Framework 4.7.2
- **UI**: Windows Forms (WinForms)
- **Project**: `BubbleApp.sln`

6.3 Main Dependencies

```
<!-- BubbleApp.csproj -->
<Reference Include="System.Windows.Forms" />
<Reference Include="System.Drawing" />
<Reference Include="System.Data" />
<PackageReference Include="Microsoft.Web.WebView2" Version="1.0.2903.40" />
```

6.4 Project Structure

```
Escritorio/BubbleApp/
├── BubbleApp.sln
├── BubbleApp/
│   ├── Formularios/
│   │   ├── FormularioInicioSesion.cs        (Login Form)
│   │   ├── FormularioPrincipal.cs           (Main Window)
│   │   ├── FormularioPerfil.cs              (User Profile)
│   │   ├── FormularioRegistro.cs            (Registration)
│   │   └── *.designer.cs                    (Designer files)
│   ├── Controles/
│   │   └── ControlMapaRadar.cs              (Custom Radar Map)
│   ├── Modelos/
│   │   ├── ModeloBurbuja.cs                 (Bubble model)
│   │   ├── ModeloUsuario.cs                 (User model)
│   │   └── ModeloPerfilUsuario.cs           (User Profile model)
│   ├── Servicios/
│   │   └── ApiCliente.cs                    (HTTP API Client)
│   ├── Estado/
│   │   └── SesionActual.cs                  (Current session)
│   ├── Programa.cs                          (Main entry point)
```

```
│   ├── App.config
│   ├── BubbleApp.csproj
│   └── bin/ & obj/
└── .vs/                                    (Visual Studio metadata)
```

6.5 Main Forms

6.5.1 SessionLoginForm

- Login/registration screen
- Fields: Email, Password
- Buttons: "Login", "Register"
- Local validations + API calls

6.5.2 MainForm

- Main application window
- Custom map (RadarMapControl)
- List of nearby bubbles
- Menu to create new bubbles
- User information (profile)

6.5.3 FormProfile

- View user data
- Edit name, avatar
- View history of created bubbles
- Logout button

6.5.4 RegistrationForm

- Registration form
- Fields: Name, Email, Password, Confirm Password
- Validations
- Saved to DB via API

6.6 ApiClient (Service)

HTTP client to communicate with Laravel API:

```
public class ApiCliente {
    private const string BaseUrl = "http://localhost:8000/api";
    private string? _token;

    public async Task<LoginResponse> Login(string email, string password);
    public async Task<RegisterResponse> Register(string name, string email, string
password);
    public async Task<List<Bubble>> ObtenerBurbujas();
    public async Task<Bubble> CrearBurbuja(double lat, double lng, string mensaje);
    public async Task<User> ObtenerPerfil();
    public async Task<bool> ActualizarPerfil(string name, string avatar);
    public async Task Logout();
}
```

6.7 Application Flow

```
Inicio
   │
   ├─ Verificar sesión guardada (token)
   │
   ├─ Sí → FormularioPrincipal
   │
   └─ No → FormularioInicioSesion
            │
            ├─ Login exitoso → FormularioPrincipal
            │
            └─ Registrarse → FormularioRegistro → FormularioPrincipal
```

6.8 ControlMapRadar

Custom control that represents:

- User position (center)
- Nearby bubbles (dots on radius)
- Interactivity (zoom, pan)
- Colors by type or age

---

7. Security

7.1 Main Security Mechanisms

Mechanism	Description
Sanctum Authentication	API tokens for desktop/mobile + Sessions for web
BCrypt Hashing	Hashed passwords with 10 rounds minimum
HTTPS/TLS	Mandatory SSL certificate in production
SQL Injection Prevention	Eloquent ORM with prepared statements
CSRF Protection	CSRF tokens in web forms
Password Validation	Unique email, password min 8 characters
Rate Limiting	Request limit per IP/user

7.2 Authentication

API (Desktop/Mobile):

- Login: send email + password
- Server responds with token
- Upcoming requests include: `Authorization: Bearer {TOKEN}`
- Tokens stored in table `personal_access_tokens`

Website:

- Session-based (no tokens)
- Session cookie after login

- Middleware `auth` protects routes

7.3 Sensitive Data

- **Passwords**: Hashed with BCrypt
- **Tokens**: Stored hashed in DB
- **Sessions**: Secure file with restricted permissions
- **API Keys**: Save to `.env` (never in code)

Model validations:

```
protected $fillable = ['name', 'email', 'password'];
protected $hidden = ['password', 'remember_token'];
```

7.4 Production Configuration

```
# .env
APP_ENV=production
APP_DEBUG=false
SESSION_SECURE_COOKIES=true
SANCTUM_STATEFUL_DOMAINS=bubble.com
```

---

8. Deployment

8.1 Local Execution

8.1.1 Prerequisites

Backend (API):

- PHP 8.2+
- composer
- SQLite3
- Node.js + npm (for Vite)

Mobile (Android):

- Android Studio
- Android SDK (API 24-36)
- Gradle

Desktop (.NET):

- Visual Studio 2019+ or Visual Studio Code
- .NET Framework 4.7.2 (Windows)

8.1.2 Backend Installation

```
# 1. Navegar a carpeta API
cd bd/bubble-api

# 2. Instalar dependencias PHP
composer install
```

```
# 3. Copiar archivo .env
cp .env.example .env

# 4. Generar clave de aplicación
php artisan key:generate

# 5. Ejecutar migraciones
php artisan migrate

# 6. (Opcional) Ejecutar seeders
php artisan db:seed

# 7. Iniciar servidor de desarrollo
php artisan serve

# Servidor activo en: http://localhost:8000
```

Alternatively with Vite:

```
npm install
npm run dev
```

8.1.3 Mobile Installation (Android)

```
# 1. Abrir en Android Studio
Open > Movil/Bubble

# 2. Sincronizar Gradle
Build > Make Project

# 3. Conectar dispositivo/emulador Android
# Dispositivo real: Activar USB Debugging
# Emulador: Crear desde Device Manager

# 4. Ejecutar app
Run > Run 'app' (Shift+F10)
```

8.1.4 Desktop Installation (.NET)

```
# 1. Abrir solución
Escritorio/BubbleApp/BubbleApp.sln

# 2. En Visual Studio
Build > Build Solution

# 3. Ejecutar
Debug > Start Debugging (F5)
```

8.2 Production Deployment

Backend API on server (VPS/Hosting):

1. SSH to server
2. Clone repository: `git clone ...`
3. Install dependencies: `composer install`
4. Configure `.env` with MySQL/PostgreSQL DB
5. Generate key: `php artisan key:generate`
6. Migrations: `php artisan migrate`
7. Configure web server (Nginx or Apache)
8. Enable HTTPS with Let's Encrypt

Database: Switch from SQLite to MySQL/PostgreSQL in production

Domain: Point DNS to server IP

8.3 Change API URL by Environment

Mobile Android (`ApiClient.kt`):

```
private const val BASE_URL =
    if (BuildConfig.DEBUG) "http://localhost:8000/api"
    else "https://api.bubble.com/api"
```

Desktop .NET (`App.config`):

```
<add key="ApiBaseUrl" value="https://api.bubble.com/api" />

---

## 9. Pruebas

### 9.1 Pruebas Backend (PHPUnit)

```bash
php artisan test
```
```

Ejecuta pruebas unitarias e integración en `tests/Feature/`

9.2 Pruebas API (Postman/Insomnia)

Importar colección con endpoints:

- POST /api/desktop/login (credenciales válidas/inválidas)
- POST /api/desktop/register (registro)
- GET /api/burbujas (listar)
- POST /api/desktop/bubble (crear - con token)
- DELETE /api/desktop/bubble (eliminar)

9.3 Pruebas Móvil (Android)

En Android Studio:

```
Run Tests > ▶ Button
```

9.4 Pruebas Escritorio (.NET)

En Visual Studio:

```
Test > Run All Tests
```

---

10. Guía de Uso y Mantenimiento

10.1 Inicio Rápido

```
# Terminal 1: API
cd bd/bubble-api
php artisan serve
# http://localhost:8000

# Terminal 2: Android Studio (Mobile)
# Open Mobile/Bubble > Run

# Terminal 3: Visual Studio (.NET Desktop)
# Open Desktop/BubbleApp/BubbleApp.sln > Run
```

10.2 Problemas Comunes

Problema	Solución
API no responde	`php artisan serve` en `bd/bubble-api`
Error 401 Unauthorized	Re-autenticarse (login nuevamente)
Error 422 Validation	Verificar datos requeridos
GPS no funciona (Móvil)	Activar GPS en dispositivo/emulador
Base de datos bloqueada	Cerrar otros procesos de Laravel
.NET no compila	Instalar .NET Framework 4.7.2

10.3 Agregar Nuevas Funcionalidades

Nuevo endpoint API:

```
php artisan make:controller Api/NewControllerFunctionality
# Add routes in routes/api.php
# Implement methods in the controller
```

Nueva pantalla Móvil:

```
// Create Activity in Android Studio
// Register in AndroidManifest.xml
// Navigate from another screen
```

Nuevo formulario Escritorio:

```
// Create form in Visual Studio (New Form)
// Add logic in Program.cs
// Compile and run
```

---

11. Mejoras Futuras

11.1 Funcionalidades Próximas

1. **Notificaciones en Tiempo Real**
   - Push notifications (móvil)
   - WebSocket para actualizaciones
   - Email notifications
2. **Búsqueda y Filtrado**
   - Filtrar por distancia, fecha
   - Búsqueda por palabras clave
   - Historial de búsquedas
3. **Interacción Social**
   - Likes/reacciones
   - Comentarios
   - Seguir/Bloquear usuarios
4. **Multimedia**
   - Imágenes en burbujas
   - Galería de usuario
   - Compresión automática
5. **Internacionalización**
   - Soporte multiidioma
   - Traducciones (ES, EN, FR)

11.2 Optimizaciones

- **Backend**: Redis cache, índices BD, async jobs
- **Móvil**: Lazy loading, batería optimizada
- **Desktop**: Caché local, renderizado async
- **Web**: Code splitting, PWA, Service Workers
- **General**: CDN, logging centralizado, APM

📚 Referencias

- **Laravel**: https://laravel.com/docs
- **Android**: https://developer.android.com
- **.NET**: https://docs.microsoft.com/dotnet
- **Google Maps**: https://developers.google.com/maps

✅ Historial

Versión	Fecha	Cambios
1.0	2026-03-29	Documentación inicial

Fin de Documentación